

# Laboratório 2 — Processamento de Bases de Dados em Lote

Bruce Trevisan - RA:277200

**1. Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada ( $ARRIVAL\_DELAY > 10$ )?**

As estatísticas para a determinação do percentual de vôos atrasados são o número de vôos atrasados e o tamanho da amostra (número total de vôos).

**2. Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dplyr`):**

- Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: AA, DL, UA e US;
- Remova observações que tenham valores faltantes em campos de interesse;
- Agrupe o conjunto de dados resultante de acordo com: dia, mês e cia. aérea;
- Para cada grupo em b., determine as estatísticas suficientes apontadas no item 1. e os retorne como um objeto da classe tibble;
- A função deve receber apenas dois argumentos: `input`: o conjunto de dados (referente ao lote em questão); `pos`: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir na função, este argumento não será empregado internamente. É importante observar que, sem a definição deste argumento, será impossível fazer a leitura por partes.

```
getStats <- function(input, pos) {  
  input %>%  
    filter(AIRLINE %in% c("AA", "DL", "UA", "US")) %>%  
  
    filter(!is.na(MONTH), !is.na(DAY), !is.na(AIRLINE), !is.na(ARRIVAL_DELAY)) %>%
```

```

group_by(DAY, MONTH, AIRLINE) %>%

summarise(
  n_atrasos = sum(ARRIVAL_DELAY > 10),
  n_total   = n(),
  .groups = "drop"
) %>%
as_tibble()
}

```

**3. Utilize alguma função readr::read\_\*\*\*\_chunked para importar o arquivo flights.csv.zip.**

- Configure o tamanho do lote (chunk) para 100 mil registros;
- Configure a função de callback para instanciar DataFrames aplicando a função getStats criada acima;
- Configure o argumento col\_types de forma que ele leia, diretamente do arquivo, apenas as colunas de interesse (veja nota de aula para identificar como realizar esta tarefa);

```

col_spec <- cols_only(
  MONTH      = col_integer(),
  DAY        = col_integer(),
  AIRLINE     = col_character(),
  ARRIVAL_DELAY = col_double()
)

stats_por_lote <- read_csv_chunked(
  file       = "./flights.csv.zip",
  callback    = DataFrameCallback$new(getStats),
  chunk_size  = 100000,
  col_types   = col_spec
)

```

**4. Crie uma função chamada computeStats que:**

- Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea);
- Retorne as informações em um tibble contendo apenas as seguintes colunas:

Cia: sigla da companhia aérea; Data: data, no formato AAAA-MM-DD (dica: utilize o comando `as.Date`); Perc: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo [0,1]

```
computeStats <- function(df) {  
  df %>%  
    group_by(AIRLINE, MONTH, DAY) %>%  
    summarise(  
      total_atrasos = sum(n_atrasos),  
      total_voos    = sum(n_total),  
      .groups = "drop"  
    ) %>%  
    mutate(  
      Perc = total_atrasos / total_voos,  
      Data = as.Date(sprintf("2015-%02d-%02d", MONTH, DAY))  
    ) %>%  
    select(Cia = AIRLINE, Data, Perc)  
}  
  
stats_final <- computeStats(stats_por_lote)
```

## 5. Produza um mapa de calor em formato de calendário para cada Cia. Aérea.

- Instale e carregue os pacotes `ggcal` e `ggplot2`.
- Defina uma paleta de cores em modo gradiente. Utilize o comando `scale_fill_gradient`. A cor inicial da paleta deve ser `#4575b4` e a cor final, `#d73027`. A paleta deve ser armazenada no objeto `pal`.
- Crie uma função chamada `baseCalendario` que recebe 2 argumentos a seguir: `stats` (tibble com resultados calculados na questão 4) e `cia` (sigla da Cia. Aérea de interesse). A função deverá:
- Criar um subconjunto de `stats` de forma a conter informações de atraso e data apenas da Cia. Aérea dada por `cia`. Para o subconjunto acima, montar a base do calendário, utilizando `ggcal(x, y)`. Nesta notação, `x` representa as datas de interesse e `y`, os percentuais de atraso para as datas descritas em `x`.
- Retornar para o usuário a base do calendário criada acima.
- Executar a função `baseCalendario` para cada uma das Cias. Aéreas e armazenar os resultados, respectivamente, nas variáveis: `cAA`, `cDL`, `cUA` e `cUS`.

- Para cada uma das Cias. Aéreas, apresente o mapa de calor respectivo utilizando a combinação de camadas do ggplot2. Lembre-se de adicionar um título utilizando o comando ggtitle. Por exemplo, cXX + pal + ggtitle("Título").

```
if (!requireNamespace("remotes", quietly = TRUE)) install.packages("remotes")
if (!requireNamespace("ggcal", quietly = TRUE)) remotes::install_github("jayjacobs/ggcal")
library(ggcal)

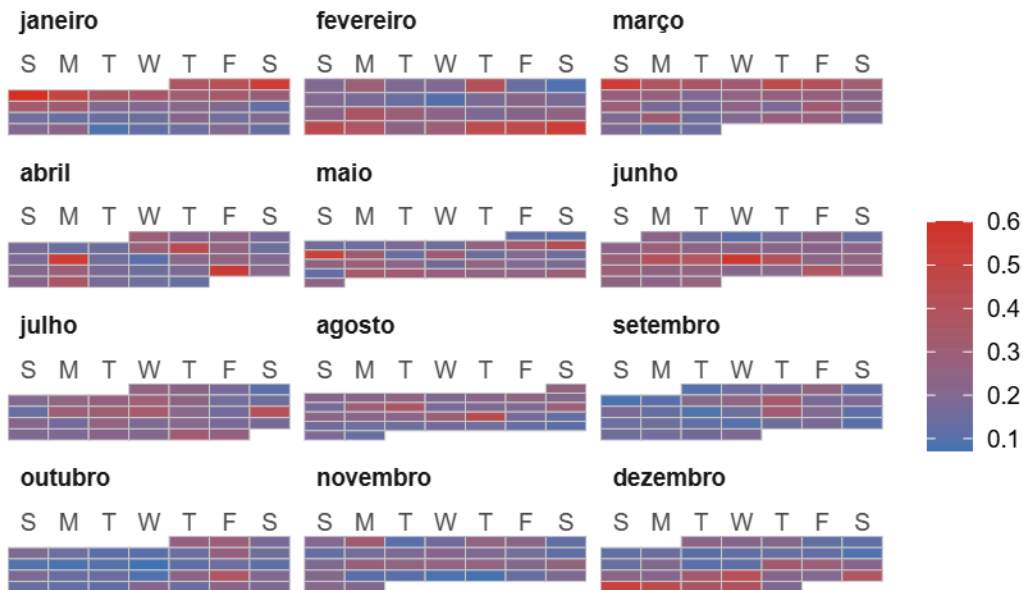
pal <- scale_fill_gradient(low = "#4575b4", high = "#d73027")

baseCalendario <- function(stats, cia) {
  stats_cia <- filter(stats, Cia == cia)
  ggcal(stats_cia$Data, stats_cia$Perc)
}
```

```
cAA <- baseCalendario(stats_final, "AA")
cDL <- baseCalendario(stats_final, "DL")
cUA <- baseCalendario(stats_final, "UA")
cUS <- baseCalendario(stats_final, "US")
```

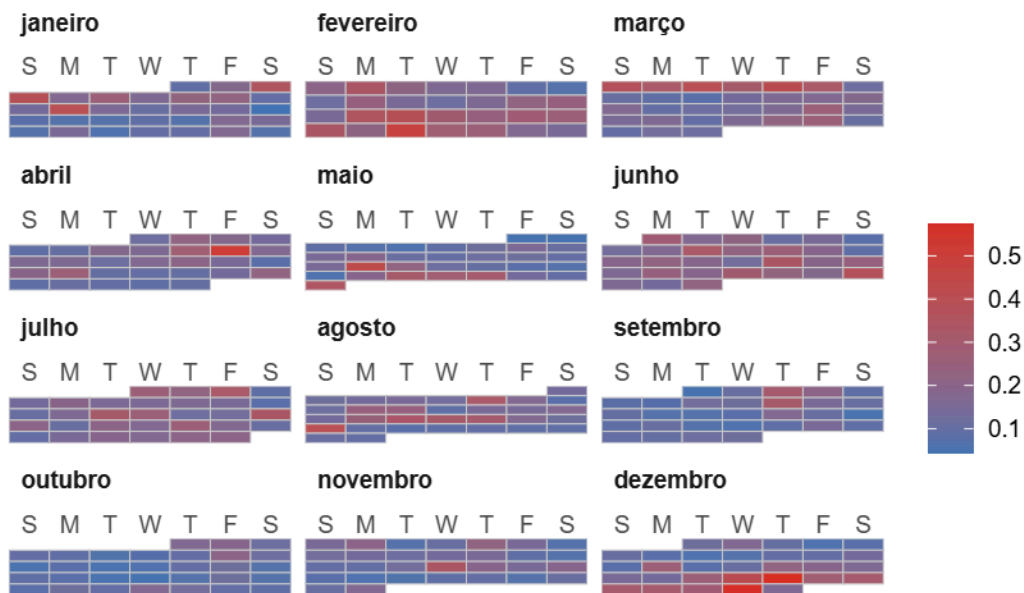
```
cAA + pal + ggtitle("Percentual de Atrasos - American Airlines (AA)")
```

### Percentual de Atrasos – American Airlines (AA)



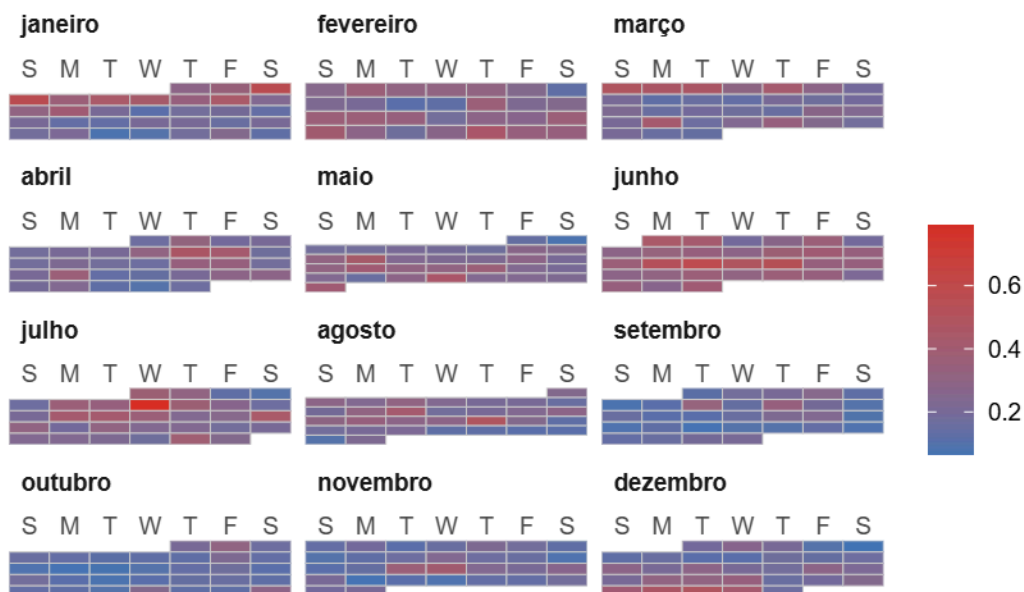
```
cDL + pal + ggtitle("Percentual de Atrasos - Delta Air Lines (DL)")
```

## Percentual de Atrasos – Delta Air Lines (DL)



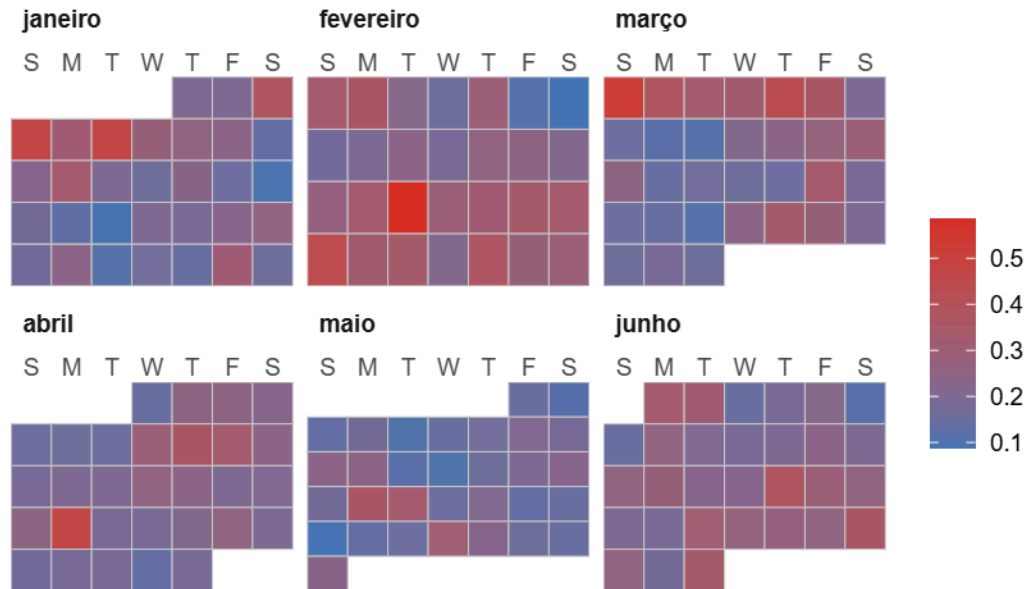
```
cUA + pal + ggtitle("Percentual de Atrasos - United Airlines (UA)")
```

## Percentual de Atrasos – United Airlines (UA)



```
cUS + pal + ggtitle("Percentual de Atrasos - US Airways (US)")
```

## Percentual de Atrasos - US Airways (US)



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-24 16:34:16 -03"
```